Agent-Core
模块架构设计文档

pi-mono-java | 2026-03-29 | Draft

模块路径	modules/agent-core
Java 版本	21 (Virtual Threads)
文档风格	Google Design Doc
目标读者	团队内部开发者

目录

1. 概述
1.1 背景与动机
1.2 设计目标
1.3 非目标
2. 系统架构
2.1 模块依赖关系
2.2 包结构总览
3. 核心设计
3.1 Agent 门面（Facade）
3.2 核心类图
3.3 执行循环时序图
3.4 事件层级
3.5 工具执行管线
3.6 消息队列
3.7 取消机制
3.8 上下文转换
3.9 代理配置
4. 并发模型
5. 外部依赖
6. 错误处理策略
7. 扩展指南
8. 测试策略
9. 术语表

1. 概述

1.1 背景与动机

agent-core 是 pi-mono-java 单体仓库中的核心模块，提供了一个通用的 AI Agent 执行引擎。它将 LLM 调用、工具执行、状态管理、事件分发等关注点解耦为独立的子系统，使上层应用（如 coding-agent-cli、TUI 终端界面）只需关注业务逻辑，而不必处理 Agent 生命周期的复杂性。

1.2 设计目标

- 可扩展的工具系统：通过 AgentTool 接口和 Hook 机制，支持任意工具的注册、校验、拦截和执行
- 流式响应：基于 Reactor 的 SSE 流式处理，支持实时文本输出和 Thinking 内容
- 线程安全：利用 Java 21 虚拟线程 + ReentrantReadWriteLock 实现高并发下的安全状态访问
- 可观测性：通过密封事件层级（Sealed Event Hierarchy）提供完整的生命周期可观测能力
- LLM 无关性：通过 StreamFunction 抽象解耦具体的 LLM 提供商

1.3 非目标

- 不负责具体工具的实现（如文件读写、代码执行等），这些由上层模块提供
- 不负责 UI 渲染，仅通过事件通知上层
- 不直接管理会话持久化，消息历史由调用方维护

2. 系统架构

2.1 模块依赖关系

agent-core 位于应用层（CLI/TUI）与 LLM 集成层（pi-ai）之间，向上提供统一的 Agent API，向下通过 api 依赖使用 pi-ai 的 LLM 抽象。

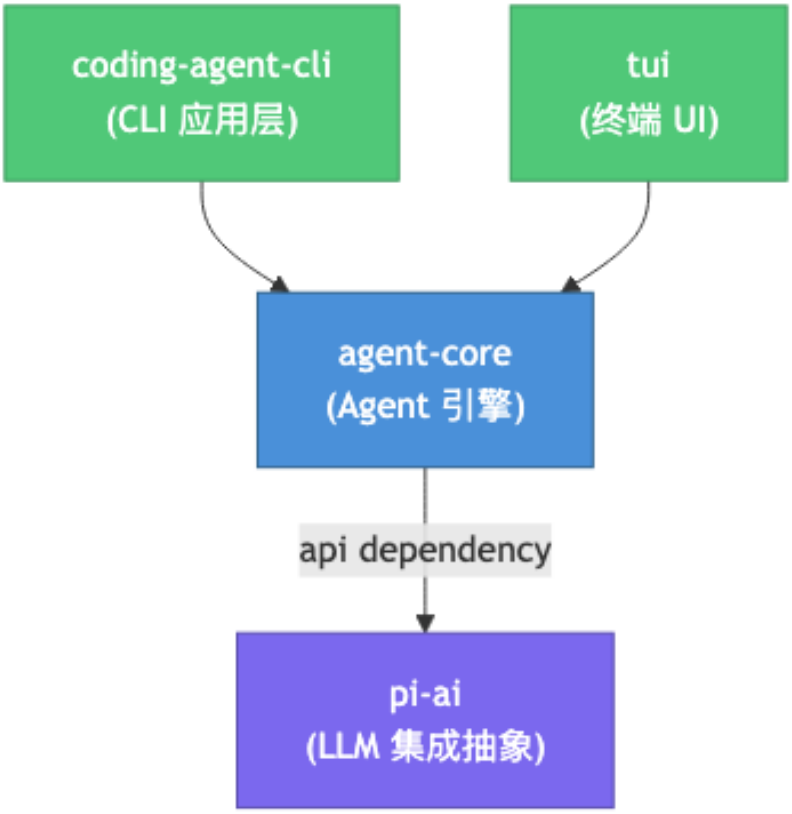

图 2-1：模块依赖关系图

2.2 包结构总览

包名	核心类	职责
com.mariozechner.pi.agent	Agent.java, PiAgentCore.java	主门面与 Spring 入口
agent.loop	AgentLoop, AgentLoopConfig, StreamFunction, SteeringMessageSupplier	核心执行循环
agent.state	AgentState, AgentStateSnapshot	线程安全状态管理
agent.event	AgentEvent (sealed) + 10 个具体事件	事件系统
agent.tool	AgentTool, ToolExecutionPipeline, CancellationToken, Hook 接口	工具执行系统
agent.context	ContextTransformer, MessageConverter	上下文转换

包名	核心类	职责
agent.queue	MessageQueue	双队列消息注入
agent.proxy	ProxyConfig	HTTP/SOCKS5 代理配置

3. 核心设计

3.1 Agent 门面（Facade）

Agent 类是模块的唯一公开 API，标注为 Spring @Service，采用 Facade 模式将内部子系统（Loop、State、Queue、Pipeline）隐藏在单一入口之后。通过 executionLock 保证同一时刻只有一个执行循环在运行。

方法	说明
prompt(message)	发送用户消息并启动执行循环，返回 CompletableFuture
continueExecution()	无新消息的情况下继续执行
abort()	取消当前正在执行的循环
waitForIdle()	阻塞等待直到 Agent 空闲
addEventListener(listener)	注册事件监听器

3.2 核心类图

以下类图展示了 Agent 模块的核心类及其关系。Agent 作为门面委托 AgentLoop 执行，依赖 AgentState 管理状态，通过 MessageQueue 接收动态消息，并使用 ToolExecutionPipeline 执行工具调用。

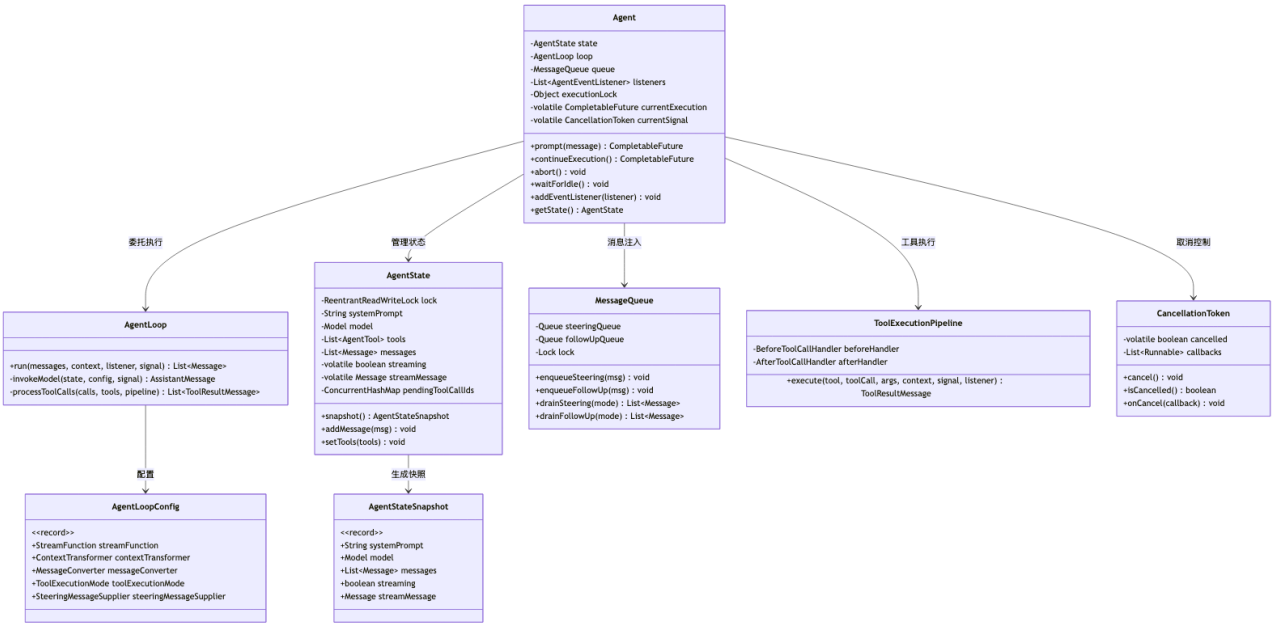

图 3-1：核心类关系图

3.3 执行循环时序图

AgentLoop 是核心执行引擎，在虚拟线程中运行。每个 Turn 包含一次 LLM 调用和可能的工具执行。循环在以下条件下继续：存在工具调用需要执行，或存在引导/追加消息需要处理。

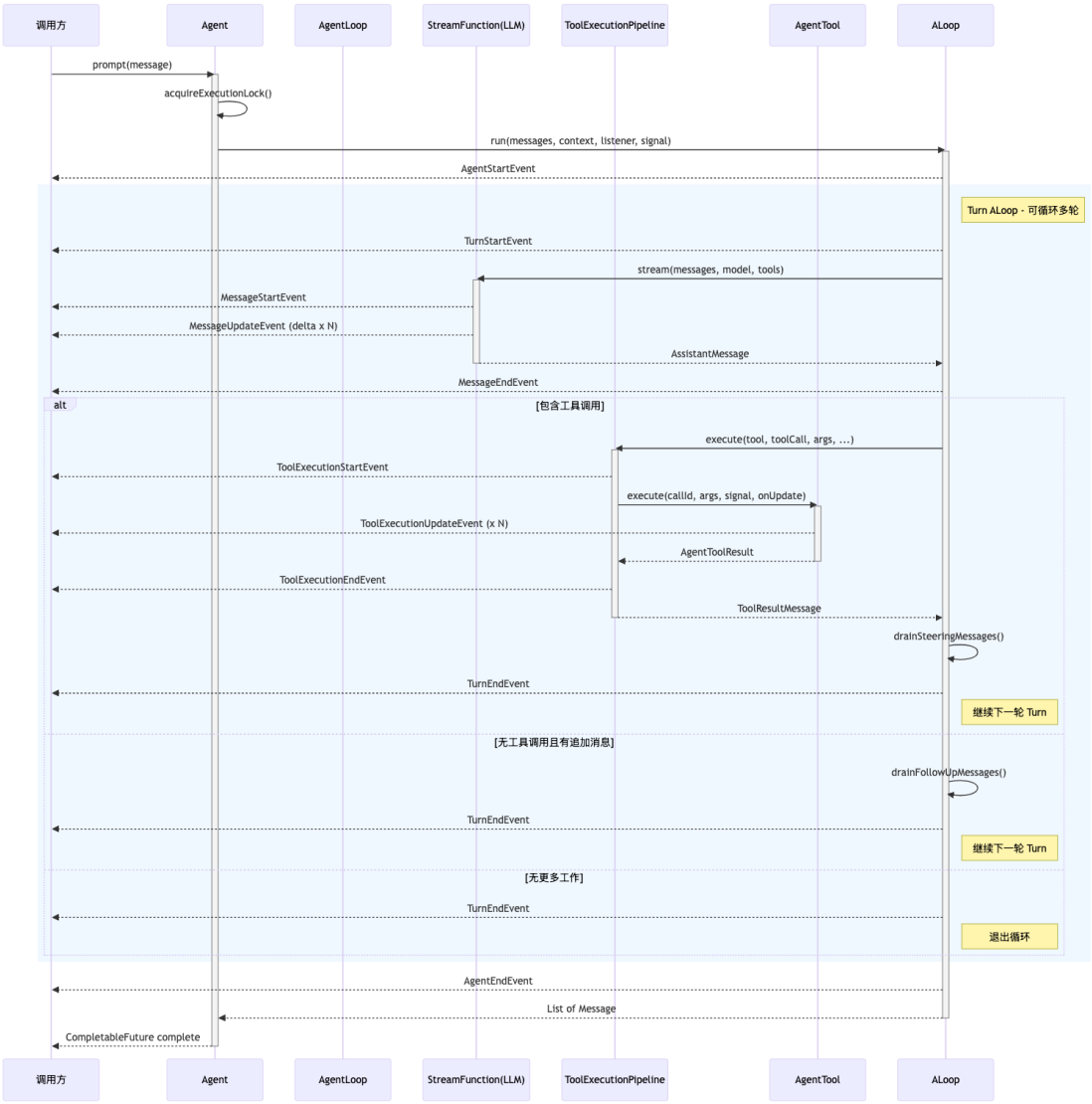

图 3-2：Agent 执行循环时序图

3.3.1 关键配置项（AgentLoopConfig）

字段	类型	说明
streamFunction	StreamFunction	可替换的 LLM 调用实现
contextTransformer	ContextTransformer	发送前的消息预处理
messageConverter	MessageConverter	消息格式适配
toolExecutionMode	ToolExecutionMode	SEQUENTIAL 或 PARALLEL
steeringMessageSupplier	SteeringMessageSupplier	回调式引导消息注入

3.4 事件层级

采用密封接口（Sealed Interface）定义事件层级，配合 Jackson 多态序列化。三层粒度（Agent → Turn → Message/Tool）让上层可按需监听不同粒度的事件。

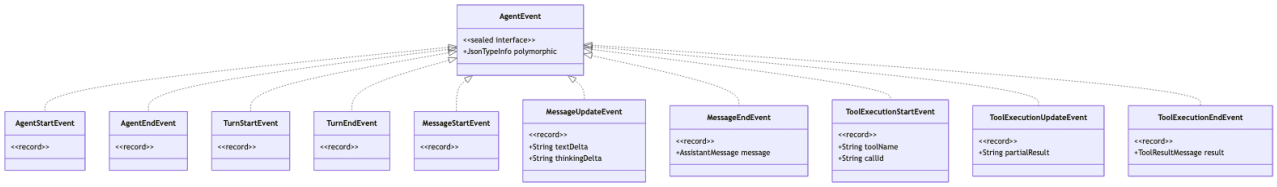

图 3-3：AgentEvent 密封事件层级

设计决策：

- 密封接口而非枚举或开放继承：编译器可验证 switch 表达式的穷尽性，同时禁止外部扩展
- CopyOnWriteArrayList 存储监听器：注册/注销不频繁，遍历频繁，读优化集合是正确选择
- 监听器异常隔离：单个监听器抛出异常不会中断 Agent 执行

3.5 工具执行管线（ToolExecutionPipeline）

工具执行管线负责工具调用的完整生命周期。前置 Hook 可拦截危险操作，后置 Hook 可覆写结果或记录审计日志。支持 JSON Schema 参数校验。

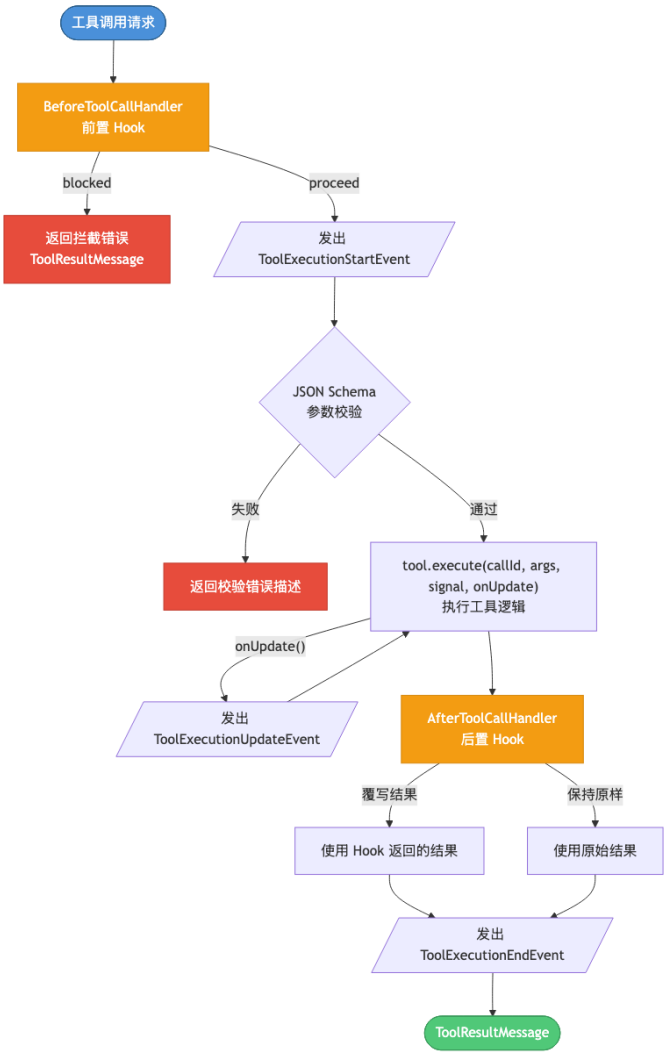

图 3-4：工具执行管线流程图

执行模式：

- SEQUENTIAL：逐个执行工具调用，适用于工具间有依赖的场景
- PARALLEL：使用 Executors.newVirtualThreadPerTaskExecutor() 并行执行，适用于独立的工具调用

Hook 机制用途：

- 权限控制（前置 Hook 拦截危险操作）
- 审计日志（后置 Hook 记录执行结果）
- 结果转换（后置 Hook 修改工具输出）

3.6 消息队列（MessageQueue）

MessageQueue 实现了双队列系统，支持在 Agent 执行过程中动态注入消息。

队列	用途	消费时机
引导队列（Steering）	在工具执行期间注入的消息	工具执行完成后、下一个 Turn 开始前
追加队列（Follow-up）	LLM 回复后追加的消息	当前 Turn 结束后、无工具调用时

投递模式：ALL（一次性消费全部）和 ONE_AT_A_TIME（每次一条）。典型场景：用户在 Agent 执行工具期间输入了新消息，该消息进入引导队列，工具执行完毕后自动作为下一轮输入。

3.7 取消机制（CancellationToken）

CancellationToken 实现了协作式取消模式。采用协作式而非强制式取消，因为工具可能持有外部资源（文件句柄、网络连接），需要机会进行清理。

- 线程安全：volatile boolean 标记取消状态
- 回调注册：onCancel(Runnable) 注册取消时的清理动作
- 传播路径：Agent.abort() → 设置 token → Loop 检查 token 退出 → 工具收到取消信号

3.8 上下文转换

接口	职责	默认实现
ContextTransformer	异步消息预处理（如截断、摘要）	无（可选配置）
MessageConverter	将内部消息格式转换为 LLM API 格式	DefaultMessageConverter（透传）

应用顺序：ContextTransformer（全局变换）→ MessageConverter（格式适配）→ StreamFunction（发送）

3.9 代理配置（ProxyConfig）

- 环境变量：HTTP_PROXY / http_proxy、HTTPS_PROXY / https_proxy、NO_PROXY / no_proxy
- 协议支持：HTTP、HTTPS、SOCKS5
- 认证支持：username:password@host:port 格式
- 安装方式：installAsDefault() 全局注入 JVM 代理设置

4. 并发模型

4.1 线程模型

组件	线程类型	说明
Agent 执行	虚拟线程	prompt() 在虚拟线程中启动 AgentLoop
并行工具执行	虚拟线程池	newVirtualThreadPerTaskExecutor()
LLM 流式响应	Reactor 调度器	非阻塞 I/O
事件监听器	调用线程（同步）	监听器在发出事件的线程上执行

4.2 同步机制

机制	使用位置	理由
ReentrantReadWriteLock	AgentState	读多写少，读锁允许并发
CopyOnWriteArrayList	事件监听器列表	注册少遍历多
ConcurrentHashMap	待处理工具调用 ID	高并发安全的 K-V 存储
volatile	streaming、streamMessage 等	轻量级可见性保证
executionLock (synchronized)	Agent.startExecution	互斥执行保证

4.3 竞态条件防护

- 单执行保证：executionLock 确保同一时刻只有一个 AgentLoop 运行
- 状态快照：外部获取的是不可变的 AgentStateSnapshot，不受后续修改影响
- 取消安全：CancellationToken 的取消状态是 volatile 的，保证跨线程可见

5. 外部依赖

依赖	版本	用途
pi-ai (内部)	—	LLM 集成抽象层（Anthropic、OpenAI、Google、AWS Bedrock）
reactor-core	(BOM)	响应式流式处理
jackson-databind	(BOM)	JSON 序列化/反序列化、事件多态支持
json-schema-validator	1.5.2	工具参数 JSON Schema 校验
spring-context	(BOM)	依赖注入（@Service）
micrometer-core	(BOM)	度量指标（已引入，尚未启用）

6. 错误处理策略

核心原则：工具层面的错误不应导致 Agent 崩溃，而是作为上下文信息反馈给 LLM，让 LLM 决定如何应对。

场景	处理方式
未知工具调用	返回描述性错误的 ToolResultMessage，不中断 Agent
工具参数校验失败	返回 Schema 校验错误信息，Agent 可重试
工具执行异常	捕获异常，包装为错误结果
Hook 异常	捕获并转换为错误结果
模型未设置	CompletableFuture.failedFuture()，快速失败
重复执行	IllegalStateException
流式调用失败	错误存入 AgentState.error，循环以 ERROR 原因退出
CompletionException	自动解包，提取根因

7. 扩展指南

7.1 实现自定义工具

```
public class MyTool implements AgentTool {
@Override
public String name() { return "my_tool"; }

@Override
public String description() { return "..."; }

@Override
public JsonNode schema() { /* JSON Schema */ }

@Override
public AgentToolResult execute(
String callId, ObjectNode args,
CancellationToken signal,
AgentToolUpdateCallback onUpdate) {
// onUpdate.onUpdate(partialResult)
// signal.isCancelled()
return new AgentToolResult("result", false);
}
}
```

7.2 注册前后置 Hook

```
// BeforeToolCallHandler
agent.setBeforeToolCallHandler(ctx -> {
if (isDangerous(ctx.toolCall()))
return BeforeToolCallResult.blocked("blocked");
return BeforeToolCallResult.proceed();
});

// AfterToolCallHandler
agent.setAfterToolCallHandler(ctx -> {
auditLog.record(ctx.toolCall(), ctx.result());
return AfterToolCallResult.useOriginal();
});
```

7.3 监听事件

```
agent.addEventListener(event -> {
switch (event) {
case MessageUpdateEvent e -> renderToUI(e.delta());
case ToolExecutionStartEvent e -> showSpinner(e.toolName());
case AgentEndEvent e -> hideSpinner();
default -> {}
}
});
```

7.4 替换 LLM 调用实现

```
var config = new AgentLoopConfig(
(messages, model, tools, signal) -> {
return myCustomLlmCall(messages, model);
},
null, // contextTransformer
new DefaultMessageConverter(),
ToolExecutionMode.PARALLEL,
null // steeringMessageSupplier
);
```

8. 测试策略

测试文件	覆盖范围
AgentTest.java	门面行为、事件发出、中止/流式
AgentLoopTest.java	核心循环、引导消息、追加消息
AgentLoopIntegrationTest.java	端到端场景
ToolExecutionPipelineTest.java	管线生命周期、Hook、错误处理
ContextConversionTest.java	消息转换
MessageQueueTest.java	队列行为
AgentStateTest.java	状态管理、并发访问
AgentEventTest.java	事件序列化

测试模式：

- Mock LLM 响应：TextResponseProvider、ToolCallProvider、ScenarioProvider 模拟不同的 LLM 行为
- Mock 工具：MockAgentTool、SteeringTool 等模拟工具执行
- 事件收集：注册监听器收集事件序列，断言事件顺序和内容

9. 术语表

术语	说明
Turn	一个回合：从发送消息到 LLM，到 LLM 回复并执行完所有工具调用
Steering Message	在工具执行期间注入的引导消息，影响下一个 Turn 的行为
Follow-up Message	在 LLM 回复后追加的消息，触发新的 Turn
Stream Function	可替换的 LLM 流式调用函数
Tool Execution Pipeline	工具调用的完整执行管线，包含校验、Hook、执行
Cancellation Token	协作式取消令牌，支持优雅中止